Customer Segmentation & Service Recommendation System

Jupyter Notebook — PDF Export

Veda Technology | Data Science Internship (Day 1 of 4)

This PDF contains the notebook's Markdown explanations and Python code cells. It is a document export of the notebook source; run the notebook in Jupyter or Google Colab to calculate metrics and generate live outputs.

Customer Segmentation & Service Recommendation System

Veda Technology — Data Science Internship (Day 1 of 4)

Project overview

This project builds a reproducible customer analytics workflow using synthetic data. It groups customers by behavior and engagement with K-Means, visualizes clusters with PCA, profiles the resulting segments, and generates transparent rule-based service or learning-program recommendations.

Business problem

A technology and learning provider may need to understand how different customers use its website, engage with content, ask for support, and purchase services. Without customer-level patterns, outreach can be generic and inefficient.

Objectives

• Create a synthetic customer dataset with at least 1,000 records.

• Inspect, clean, and explore the data.

• Engineer behavioral features and standardize numeric inputs.

• Compare candidate K-Means cluster counts using inertia (elbow method) and silhouette score.

• Profile and name segments from measured cluster characteristics.

• Recommend services with explainable rules.

• Export datasets, profiles, evaluation metrics, and recommendations.

Privacy note: This notebook generates synthetic data only. It does not use private Veda Technology customer records.

1. Import libraries and configure reproducibility

We use a fixed seed so the synthetic data and model results are reproducible in the same compatible software environment.

Python Code Cell 3

```
import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

RANDOM_SEED = 42
rng = np.random.default_rng(RANDOM_SEED)
OUTPUT_DIR = Path("outputs")
DATA_DIR = Path("data")
OUTPUT_DIR.mkdir(exist_ok=True)
DATA_DIR.mkdir(exist_ok=True)

sns.set_theme(style="whitegrid", context="notebook")
pd.set_option("display.max_columns", 50)
print("Libraries imported. Random seed:", RANDOM_SEED)
```

2. Generate a realistic synthetic dataset

The data generator creates several broad behavior patterns to make the exercise useful: engaged customers, occasional visitors, new/low-engagement customers, and customers who frequently ask about services. These are latent simulation patterns, not labels given to the clustering model. The clustering step will use only behavioral numeric features.

Python Code Cell 5

```
N_CUSTOMERS = 1200
customer_types = np.array(["Individual", "Student", "Small Business", "Professional"])
programs = np.array(["Data Science", "Generative AI", "Web Development", "Data Analytics", "Cloud &amp;amp;
    DevOps"])

# Latent groups are used only to simulate plausible behavior, not as model inputs.
latent = rng.choice(4, size=N_CUSTOMERS, p=[0.30, 0.30, 0.22, 0.18])
age = rng.integers(18, 61, size=N_CUSTOMERS)
visits_means = np.array([18, 4, 2, 9])
session_means = np.array([14, 5, 2.5, 7])
engagement_means = np.array([82, 42, 18, 55])
inquiry_means = np.array([2, 1, 0.4, 7])
purchase_means = np.array([3.0, 0.8, 0.1, 1.3])
spend_means = np.array([18000, 4500, 500, 9000])
support_means = np.array([1.5, 0.8, 0.2, 4.5])
satisfaction_means = np.array([4.4, 3.8, 3.4, 3.7])

website_visits = rng.poisson(visits_means[latent]).clip(0, 60)
session_duration = rng.gamma(shape=2.2, scale=session_means[latent] / 2.2).clip(0.3, 60)
engagement_score = rng.normal(engagement_means[latent], 11).clip(0, 100)
service_inquiries = rng.poisson(inquiry_means[latent]).clip(0, 25)
purchases = rng.poisson(purchase_means[latent]).clip(0, 15)
spending = (rng.gamma(2.0, np.maximum(spend_means[latent], 100) / 2.0)).clip(0, 150000).round(2)
support_interactions = rng.poisson(support_means[latent]).clip(0, 30)
satisfaction = rng.normal(satisfaction_means[latent], 0.55).clip(1, 5).round(1)

df = pd.DataFrame({
    "Customer_ID": [f"CUST-{i:05d}" for i in range(1, N_CUSTOMERS + 1)],
    "Age": age,
    "Customer_Type": rng.choice(customer_types, N_CUSTOMERS, p=[0.42, 0.25, 0.15, 0.18]),
    "Preferred_Program": rng.choice(programs, N_CUSTOMERS),
    "Website_Visits": website_visits,
    "Avg_Session_Minutes": session_duration.round(2),
    "Engagement_Score": engagement_score.round(1),
    "Service_Inquiries": service_inquiries,
    "Previous_Purchases": purchases,
    "Spending_Amount": spending,
    "Support_Interactions": support_interactions,
    "Satisfaction_Score": satisfaction
})

# Add a small amount of realistic missingness, then let the cleaning section handle it.
for col in ["Avg_Session_Minutes", "Engagement_Score", "Satisfaction_Score"]:
    missing_idx = rng.choice(df.index, size=round(0.015 * len(df)), replace=False)
    df.loc[missing_idx, col] = np.nan

# Add a few duplicated rows to demonstrate duplicate removal.
df = pd.concat([df, df.sample(8, random_state=RANDOM_SEED)], ignore_index=True)
print("Raw dataset shape:", df.shape)
df.head()
```

3. Inspect the data

Check sample rows, data types, descriptive statistics, missing values, and duplicate rows before deciding how to clean the dataset.

Python Code Cell 7

```
display(df.head())
df.info()
display(df.describe(include="all").T)
print("Missing values per column:")
display(df.isna().sum().sort_values(ascending=False).to_frame("missing_count"))
print("Duplicate rows:", int(df.duplicated().sum()))
```

4. Clean the dataset

Remove exact duplicate rows. Fill numeric missing values with their column medians, a simple robust baseline. Check the resulting shape and missing-value count. In a real deployment, imputation choices should be validated against business context.

Python Code Cell 9

```
clean_df = df.drop_duplicates().copy()
numeric_cols = clean_df.select_dtypes(include=np.number).columns.tolist()
clean_df[numeric_cols] = clean_df[numeric_cols].fillna(clean_df[numeric_cols].median())
clean_df["Customer_Type"] = clean_df["Customer_Type"].fillna(clean_df["Customer_Type"].mode().iloc[0])
clean_df["Preferred_Program"] =
    clean_df["Preferred_Program"].fillna(clean_df["Preferred_Program"].mode().iloc[0])
print("Cleaned dataset shape:", clean_df.shape)
print("Remaining missing values:", int(clean_df.isna().sum().sum()))
```

```
clean_df.to_csv(DATA_DIR / "synthetic_customers_cleaned.csv", index=False)
```

5. Exploratory data analysis

These plots summarize distributions and relationships. They describe this synthetic sample only; they should not be treated as measured company-wide customer facts.

Python Code Cell 11

```
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
axes[0, 0].hist(clean_df["Website_Visits"].dropna(), bins=25)
axes[0, 0].set_title("Website visits per customer")
axes[0, 1].hist(clean_df["Engagement_Score"].dropna(), bins=25)
axes[0, 1].set_title("Engagement score distribution")
sns.boxplot(data=clean_df, x="Customer_Type", y="Spending_Amount", ax=axes[1, 0])
axes[1, 0].set_title("Spending by customer type")
axes[1, 0].tick_params(axis="x", rotation=20)
sns.scatterplot(data=clean_df, x="Website_Visits", y="Engagement_Score",
                hue="Preferred_Program", alpha=.65, ax=axes[1, 1])
axes[1, 1].set_title("Visits and engagement by preferred program")
axes[1, 1].legend(fontsize=7, loc="best")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 7))
corr = clean_df.select_dtypes(include=np.number).drop(columns=[], errors="ignore").corr()
sns.heatmap(corr, cmap="vlag", center=0, annot=True, fmt=".2f")
plt.title("Numeric feature correlation matrix")
plt.tight_layout()
plt.show()
```

6. Feature engineering and feature selection

We add simple measures that reflect repeated activity and engagement efficiency. Customer_ID is never used as a model feature. Categorical fields are retained for later segment interpretation and recommendations, not fed directly into K-Means in this baseline.

Python Code Cell 13

```
feature_df = clean_df.copy()
feature_df["Visits_Per_Purchase"] = feature_df["Website_Visits"] / (feature_df["Previous_Purchases"] + 1)
feature_df["Inquiry_Rate_Per_Visit"] = feature_df["Service_Inquiries"] / (feature_df["Website_Visits"] +
    1)
feature_df["Engagement_Per_Visit"] = feature_df["Engagement_Score"] / (feature_df["Website_Visits"] + 1)

FEATURES = [
    "Age", "Website_Visits", "Avg_Session_Minutes", "Engagement_Score",
    "Service_Inquiries", "Previous_Purchases", "Spending_Amount",
    "Support_Interactions", "Satisfaction_Score", "Visits_Per_Purchase",
    "Inquiry_Rate_Per_Visit", "Engagement_Per_Visit"
]
X = feature_df[FEATURES].replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Number of model features:", len(FEATURES))
print("Customer ID included in model features?", "Customer_ID" in FEATURES)
```

7. Select a candidate cluster count

Use both inertia (the elbow method) and silhouette score. Inertia always decreases as clusters are added, so it is not enough on its own. Silhouette scores help assess separation and cohesion, but they are not a guarantee of business usefulness. We select the best silhouette among the tested values as a transparent baseline; inspect the plots and profiles before interpreting the result.

Python Code Cell 15

```
candidate_ks = range(2, 9)
inertias, silhouettes = [], []
for k in candidate_ks:
    model = KMeans(n_clusters=k, random_state=RANDOM_SEED, n_init=10)
    labels = model.fit_predict(X_scaled)
    inertias.append(model.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))

metrics_df = pd.DataFrame({
    "k": list(candidate_ks),
    "inertia": inertias,
    "silhouette_score": silhouettes
```

```
})
display(metrics_df)
metrics_df.to_csv(OUTPUT_DIR / "cluster_evaluation_metrics.csv", index=False)

fig, ax1 = plt.subplots(figsize=(8, 5))
ax1.plot(metrics_df["k"], metrics_df["inertia"], marker="o")
ax1.set_xlabel("Number of clusters (k)")
ax1.set_ylabel("Inertia")
ax1.set_title("Elbow method: inertia by k")
ax1.set_xticks(list(candidate_ks))
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(metrics_df["k"], metrics_df["silhouette_score"], marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette score")
plt.title("Silhouette score by k")
plt.xticks(list(candidate_ks))
plt.tight_layout()
plt.show()

BEST_K = int(metrics_df.loc[metrics_df["silhouette_score"].idxmax(), "k"])
print("Selected k using the highest silhouette score among tested candidates:", BEST_K)
```

8. Fit K-Means and assign segments

Fit the final model using the selected k. The assigned cluster numbers are machine-generated IDs, not ordered rankings.

Python Code Cell 17

```
kmeans = KMeans(n_clusters=BEST_K, random_state=RANDOM_SEED, n_init=10)
feature_df["Cluster"] = kmeans.fit_predict(X_scaled)
final_silhouette = silhouette_score(X_scaled, feature_df["Cluster"])
print(f"Final silhouette score: {final_silhouette:.4f}")
```

9. PCA visualization

PCA projects standardized feature vectors into two dimensions for visualization. The plot can show broad group separation, but the two plotted components do not preserve every detail of the original feature space.

Python Code Cell 19

```
pca = PCA(n_components=2, random_state=RANDOM_SEED)
coords = pca.fit_transform(X_scaled)
feature_df["PCA1"] = coords[:, 0]
feature_df["PCA2"] = coords[:, 1]
print("Explained variance by two components:", round(float(pca.explained_variance_ratio_.sum()), 4))

plt.figure(figsize=(10, 7))
sns.scatterplot(data=feature_df, x="PCA1", y="PCA2", hue="Cluster", palette="tab10", alpha=.7)
plt.title("Customer clusters projected into PCA space")
plt.xlabel("Principal component 1")
plt.ylabel("Principal component 2")
plt.legend(title="Cluster")
plt.tight_layout()
plt.show()
```

10. Cluster characteristics and segment profiles

Summarize average behavior for each cluster. Segment names are assigned from observed cluster means using transparent heuristic rules; review the resulting profile table before using the names in a business setting.

Python Code Cell 21

```
profile_cols = [
    "Website_Visits", "Avg_Session_Minutes", "Engagement_Score", "Service_Inquiries",
    "Previous_Purchases", "Spending_Amount", "Support_Interactions", "Satisfaction_Score"
]
profiles = feature_df.groupby("Cluster")[profile_cols].mean().round(2)
profiles["Customer_Count"] = feature_df.groupby("Cluster")["Customer_ID"].count()
profiles["Customer_Percent"] = (100 * profiles["Customer_Count"] / len(feature_df)).round(1)

def name_segment(row, medians):
    if row["Engagement_Score"] &gt;= medians["Engagement_Score"] and row["Website_Visits"] &gt;=
    medians["Website_Visits"]:
        return "Highly Engaged Customers"
    if row["Service_Inquiries"] &gt;= medians["Service_Inquiries"] and row["Support_Interactions"] &gt;=
    medians["Support_Interactions"]:
        return "Service-Focused Customers"
    if row["Website_Visits"] &lt;= medians["Website_Visits"] and row["Engagement_Score"] &lt;=
    medians["Engagement_Score"]:
```

```
        return "New or Low-Engagement Customers"
    return "Occasional Visitors"

overall_medians = feature_df[profile_cols].median()
profiles["Segment_Name"] = profiles.apply(lambda row: name_segment(row, overall_medians), axis=1)
cluster_to_name = profiles["Segment_Name"].to_dict()
feature_df["Segment_Name"] = feature_df["Cluster"].map(cluster_to_name)
display(profiles.sort_index())
profiles.to_csv(OUTPUT_DIR / "customer_segment_profiles.csv")
```

11. Rule-based service and learning recommendations

Recommendations are intentionally explainable and can be adjusted by a business stakeholder. These are illustrative mappings for a synthetic dataset, not official Veda Technology product offerings. The preferred program is used to personalize the suggested learning path.

Python Code Cell 23

```
def recommend_customer(row):
    segment = row["Segment_Name"]
    program = row["Preferred_Program"]
    if segment == "Highly Engaged Customers":
        service = "Advanced project mentoring or an advanced learning pathway"
        reason = "High visits and engagement suggest readiness for deeper learning."
    elif segment == "Service-Focused Customers":
        service = "Personalized consultation and guided onboarding"
        reason = "Frequent inquiries/support interactions suggest that guidance may help."
    elif segment == "New or Low-Engagement Customers":
        service = "Beginner orientation, starter resources, and a short introductory workshop"
        reason = "A low-friction introduction may help the customer discover relevant value."
    else:
        service = "Targeted webinar and a curated intermediate learning pathway"
        reason = "A focused next step may encourage more consistent engagement."
    if row["Satisfaction_Score"] &lt; 3:
        service += " + follow-up to understand satisfaction concerns"
        reason += " Satisfaction is low, so follow-up should come before promotional outreach."
    return pd.Series({
        "Recommended_Service": service,
        "Recommendation_Reason": reason,
        "Recommended_Program": program
    })

recommendations = feature_df.apply(recommend_customer, axis=1)
feature_df = pd.concat([feature_df, recommendations], axis=1)
display(feature_df[[
    "Customer_ID", "Segment_Name", "Preferred_Program", "Recommended_Service",
    "Recommended_Program", "Recommendation_Reason"
]].head(10))
```

12. Export project outputs

The notebook exports the cleaned dataset, customer-level segments and recommendations, cluster profiles, and model evaluation metrics. The output files are created when this notebook is executed from the project root.

Python Code Cell 25

```
clean_df.to_csv(OUTPUT_DIR / "cleaned_customers.csv", index=False)
feature_df.to_csv(OUTPUT_DIR / "customer_segments_and_recommendations.csv", index=False)
profiles.to_csv(OUTPUT_DIR / "customer_segment_profiles.csv")
metrics_df.to_csv(OUTPUT_DIR / "cluster_evaluation_metrics.csv", index=False)
print("Exported files:")
for path in sorted(OUTPUT_DIR.glob("*.csv")):
    print("-", path, f"({path.stat().st_size:,} bytes)")
```

13. Business insights and actionable recommendations

Use the calculated tables above to make decisions. Do not assume that a segment name alone proves a customer need.

Suggested interpretation workflow:

1. Compare each segment's engagement, visits, inquiries, spending, and satisfaction against the overall median.
2. Prioritize helpful education and support rather than aggressive marketing.
3. Test recommendations with a small, opt-in pilot and measure click-through, enrollment, satisfaction, and opt-out rates.
4. Revisit segments periodically because customer behavior changes.
5. Check fairness and usefulness across customer types and age ranges before operational use.

14. Limitations

• The dataset is synthetic and cannot establish real Veda Technology customer behavior.

• K-Means favors roughly spherical clusters and is sensitive to scaling, outliers, and chosen k.

• The silhouette score is only one clustering diagnostic; interpretability and business utility matter too.

• Segment labels are heuristic summaries, not ground-truth customer categories.

• Rule-based recommendations are not causal and have not been validated with real outcomes.

• The baseline uses numerical features; categorical preferences are used only for interpretation/recommendation.

15. Conclusion

This project demonstrates an end-to-end customer segmentation workflow: synthetic data generation, cleaning, EDA, feature engineering, scaling, cluster evaluation, K-Means, PCA visualization, segment profiling, explainable recommendations, and CSV exports. Before production use, validate the approach on authorized real data, measure recommendation outcomes, and add monitoring and privacy controls.

16. Interview questions and answers

1. What is customer segmentation? Grouping customers with similar characteristics or behaviors to tailor services and communication.

2. Why use K-Means? It is a simple, scalable baseline for grouping numeric feature vectors.

3. Why standardize features? K-Means uses distances; features with large numeric scales can otherwise dominate.

4. Why exclude Customer_ID? It is an identifier, not meaningful customer behavior, and can distort distances.

5. What is inertia? The sum of squared distances from samples to their assigned cluster centers.

6. What is the elbow method? A visual heuristic for finding where extra clusters yield diminishing inertia improvement.

7. What does silhouette score measure? How similar a point is to its own cluster versus nearby alternative clusters; higher is generally better.

8. What is PCA? A dimensionality-reduction method that projects data onto directions of high variance.

9. Does PCA improve clustering automatically? No. Here it is used for visualization after clustering on standardized original features.

10. Why handle missing values? Many estimators cannot accept missing numeric values; imputation also makes the processing explicit.

11. What is a limitation of K-Means? It assumes distance-based, roughly spherical groups and is sensitive to outliers and k.

12. How would you evaluate recommendations? Run a controlled pilot and measure conversion, satisfaction, retention, and opt-out rates.

13. Why use synthetic data? It enables safe demonstration without exposing private customer information.

14. How could this be improved? Add robust validation, categorical encoding where appropriate, stability checks, business feedback, and monitoring.

15. What is data leakage? Using information unavailable at prediction time or target-derived information that makes evaluation unrealistically optimistic.